Compiler Construction

Teacher : Zulfiqar Ali

Email: Zulfiqar.rafique@nu.edu.pk

Office: No.6,Basement -1 (CS Block 1)

Week 7:

Bottom-up Parsing

Reduction

Handle Pruning

Shift Reduce Parsing

LR(0)

SLR

Bottom Up Parsing

A bottom-up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top). It is convenient to describe parsing as the process of building parse trees, although a front end may in fact carry out a translation directly without building an explicit tree. The sequence of tree snapshots in Fig. 4.25 illustrates

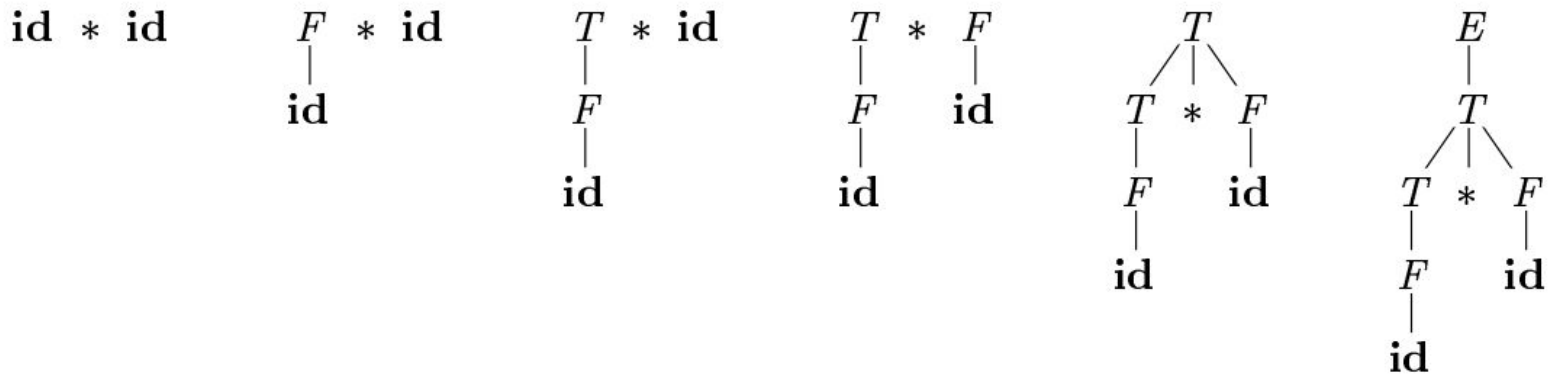


Figure 4.25: A bottom-up parse for **id * id**

a bottom-up parse of the token stream **id * id**, with respect to the expression grammar (4.1).

Reduction

We can think of bottom-up parsing as the process of "reducing" a string w to the start symbol of the grammar. At each *reduction* step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.

The key decisions during bottom-up parsing are about when to reduce and about what production to apply, as the parse proceeds.

Handle Pruning

Bottom-up parsing during a left-to-right scan of the input constructs a rightmost derivation in reverse. Informally, a "handle" is a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation.

For example, adding subscripts to the tokens **id** for clarity, the handles during the parse of $\mathbf{id}_1 * \mathbf{id}_2$ according to the expression grammar (4.1) are as in Fig. 4.26. Although T is the body of the production $E \to T$, the symbol T is not a handle in the sentential form $T * \mathbf{id}_2$. If T were indeed replaced by E, we would get the string $E * \mathbf{id}_2$, which cannot be derived from the start symbol E. Thus, the leftmost substring that matches the body of some production need not be a handle.

Handle Pruning

RIGHT SENTENTIAL FORM	HANDLE	REDUCING PRODUCTION
$\mathbf{id}_1 * \mathbf{id}_2$	$\mathbf{id}_1$	$F \to \mathbf{id}$
$F * \mathbf{id}_2$	F	$T \to F$
$T * \mathbf{id}_2$	$\mathbf{id}_2$	$F \to \mathbf{id}$
$T * F$	$T * F$	$T \to T * F$
T	T	$E \to T$

Figure 4.26: Handles during a parse of $\mathbf{id}_1 * \mathbf{id}_2$

Shift Reducing Parsing

Shift-reduce parsing is a form of bottom-up parsing in which a stack holds grammar symbols and an input buffer holds the rest of the string to be parsed. As we shall see, the handle always appears at the top of the stack just before it is identified as the handle.

We use \$ to mark the bottom of the stack and also the right end of the input. Conventionally, when discussing bottom-up parsing, we show the top of the stack on the right, rather than on the left as we did for top-down parsing. Initially, the stack is empty, and the string w is on the input, as follows:

STACK	INPUT
\$	w \$

Upon entering this configuration, the parser halts and announces successful completion of parsing. Figure 4.28 steps through the actions a shift-reduce parser might take in parsing the input string $\mathbf{id}_1 * \mathbf{id}_2$ according to the expression grammar (4.1).

Grammar
E□E+T|T
T□T*F|F
F□ID

STACK	INPUT	ACTION
\$	$\mathbf{id}_1 * \mathbf{id}_2$ \$	shift
\$ $\mathbf{id}_1$	$* \mathbf{id}_2$ \$	reduce by $F \rightarrow \mathbf{id}$
\$ F	$* \mathbf{id}_2$ \$	reduce by $T \rightarrow F$
\$ T	$* \mathbf{id}_2$ \$	shift
\$ $T *$	$\mathbf{id}_2$ \$	shift
\$ $T * \mathbf{id}_2$	\$	reduce by $F \rightarrow \mathbf{id}$
\$ $T * F$	\$	reduce by $T \rightarrow T * F$
\$ T	\$	reduce by $E \rightarrow T$
\$ E	\$	accept

Figure 4.28: Configurations of a shift-reduce parser on input $\mathbf{id}_1 * \mathbf{id}_2$

Examples

- Consider the following grammar-

- **E ⟶ E – E**
  E ⟶ E x E
  E ⟶ id

Parse the input string id – id x id using a shift-reduce parser.

Stack	Input Buffer	Parsing Action
$	id – id x id $	Shift
$ id	– id x id $	Reduce E ⟶ id
$ E	– id x id $	Shift
$ E –	id x id $	Shift
$ E – id	x id $	Reduce E ⟶ id
$ E – E	x id $	Shift
$ E – E x	id $	Shift
$ E – E x id	$	Reduce E ⟶ id
$ E – E x E	$	Reduce E ⟶ E x E
$ E – E	$	Reduce E ⟶ E – E
$ E	$	Accept

Conflicts in Shift Reduce Parsing

There are context-free grammars for which shift-reduce parsing cannot be used. Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and also the next k input symbols, cannot decide whether to shift or to reduce (a *shift/reduce conflict*), or cannot decide which of several reductions to make (a *reduce/reduce conflict*).

- Types of Conflicts
  - Shift/Reduce conflicts
  - Reduce/Reduce Conflicts

Shift Reduce Conflicts

- Grammar
  - S□aABe A□Abc|b B□d

Stact	Input String	Actoin	Issue/Conflix
$	abbcde$	Shift	
$a	bbcde$	Shift b	
$ab	bcde$	Reduce b□A	Here there is a conflict, whether we do shift next string or reduce b□A
$aA	bbcde$	Shift b	
$aAb	bcde$		Again we have shift/reduce conflict

- Here we are not sure, whether shift or reduce will lead to parse successful

Reduce/Reduce Conflicts

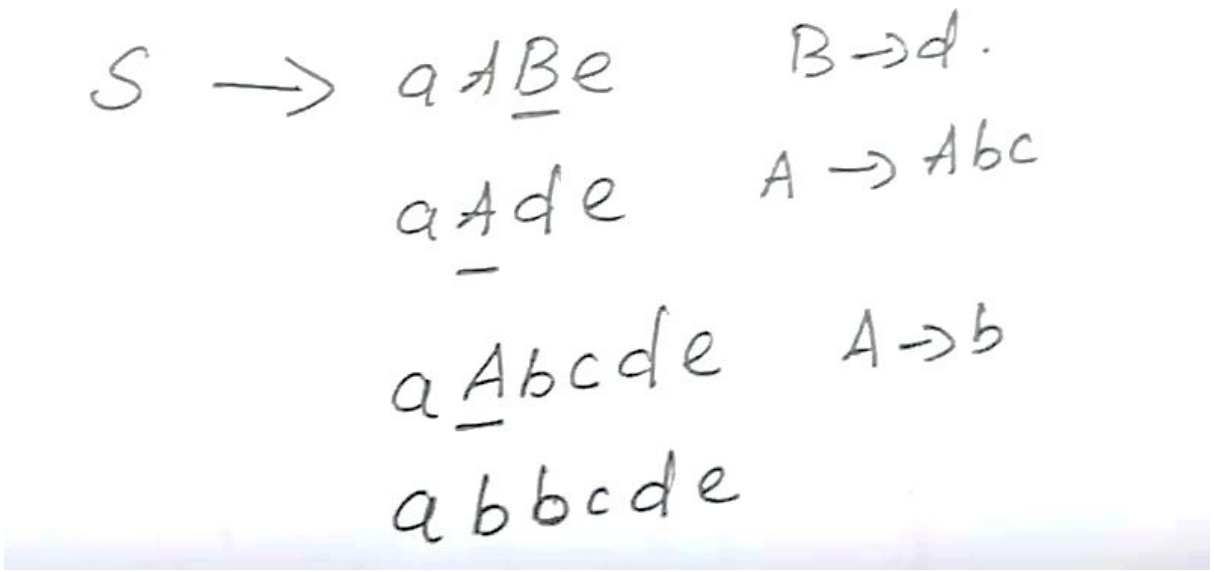
- Grammar
  - S□aABe|b A□Abc|b B□d

Stack	Input String	Action	Issue/Conflict
$	abbcde$	Shift	
$a	bbcde$	Shift b	
$ab	bbcde$	Reduce b□B	Here there is a conflict, we have 2 reduce options, we can reduce b□S also can b□A

- Here we are not sure, whether shift will be successful parsing

Solution for Conflicts

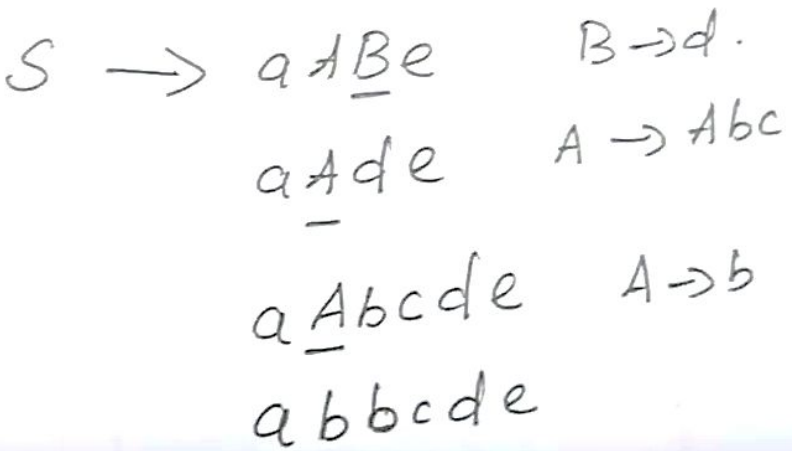
- To avoid the conflicts do Right Most Derivation RMD first, for the given input string, as per grammar.



Shift Reduce Conflicts (solved

- Grammar
  - S□aABe A□Abc|b B□d

Stact	Input String	Actoin
$	abbcde$	Shift
$a	bbcde$	Shift b
$ab	bcde$	Reduce b□A
$aA	bcde$	Shift b
$aAb	cde$	Shift c
$aAbc	de$	Reduce A□Abc
$aA	de$	Shift d
$aAd	e$	Reduce d
$aAB	e$	Shift e
$aABe	$	Reduce S□aABe
$S	$	**Accept**



LR Introduction to LR Parsing: Simple LR

- The most prevalent type of bottom-up parser today is based on a concept called LR(k) parsing;
- The LR parser is an efficient bottom-up syntax analysis technique that can be used for a large class of context-free grammar.
- L stands for the left to right scanning
- R stands for rightmost derivation in reverse
- k stands for no. of input symbols of lookahead.
- The cases k = 0 or k = 1 are of practical interest, and we shall only consider LR parsers with k <= 1 here. When (k) is omitted, k is assumed to be 1

LR Parsers

- LR parsers are table-driven, much like the non-recursive LL parsers.
- A grammar for which we can construct a parsing table using one of the methods in this section and the next is said to be an LR grammar.
- Intuitively, for a grammar to be LR it is sufficient that a left-to-right shift-reduce parser be able to recognize handles of right-sentential forms when they appear on top of the stack.
-

Why LR Parser

- LR parsers can be constructed to recognize virtually all programming language constructs for which context-free grammars can be written.
- Non LR context-free grammars exist, but these can generally be avoided for typical programming-language constructs.
- The LR-parsing method is the most general non-backtracking shift-reduce parsing method known, yet it can be implemented as efficiently.
- An LR parser can detect a syntactic error as soon as it is possible to do so on a left-to-right scan of the input.
-

Why LR Parser

- Most powerful deterministic context-free grammar parser practical for implementation.
- Can handle a wider class of grammars than LL parsers (e.g., left-recursion).
- Grammar writing is easier and more natural due to less restrictions.
- Better error detection as parsing is deferred until a reduction happens.
- Well-suited for automatic parser generation from grammar specifications.
- High parsing efficiency with O(n) time complexity for most inputs.
- Left-to-right scanning with Rightmost derivation provides optimal lookahead.
- Excellent for programming languages where grammar complexity is high.

Items and LR(0) Automation

- How does a shift-reduce parser know when to shift and when to reduce?
  - An LR parser makes shift-reduce decisions by maintaining states to keep track of where we are in a parse.
  - States represent sets of items." An LR(0) item (item for short) of a grammar G is a production of G with a dot at some position of the body.
    Thus, production A □ XYZ yields the four items.
  - The production **A □ Epsilon** generates only one item, A□.

$$A \to \cdot XYZ$$
$$A \to X \cdot YZ$$
$$A \to XY \cdot Z$$
$$A \to XYZ \cdot$$

Items and LR(0) Automation

- Intuitively, an item indicates how much of a production we have seen at a given point in the parsing process. For example, the item A ->.X Y Z indicates that we hope to see a string derivable from X Y Z next on the input.
- A -> X.Y Z indicates that we have just seen on the input a string derivable from X and that we hope next to see a string derivable from Y Z.
- Item A □ XYZ indicates that we have seen the body XY Z and that it may be time to reduce XY Z to A.

- One collection of sets of LR(0) items, called the **canonical LR(0) collection**, provides the basis for constructing a **deterministic finite automaton** that is used to make parsing decisions.
- Such an automaton is called an LR(0) automaton.
- In particular, each state of the LR(0) automaton represents a set of items in the canonical LR(0) collection..

Augmented Grammar

- To construct the canonical LR(0) collection for a grammar, we define an augmented grammar and two functions, **CLOSURE** and **GOTO**.
- If G is a grammar with start symbol S, then G', the augmented grammar for G, is G' with a new start symbol S' and production S'-> S.
- The purpose of this new starting production is to indicate to the parser when it should stop parsing and announce acceptance of the input.
- That is, acceptance occurs when and only when the parser is about to reduce by S' S.

Closure()

- The CLOSURE() operation takes a set of LR(0) items as input and expands it by adding all possible items implied by the current set. An LR(0) item is a production rule with a dot • indicating the parsing progress, such as $A \rightarrow \alpha \bullet X\beta$.
- **Rule:** If an item $A \rightarrow \alpha \bullet B\beta$ is in the set, and B is a non-terminal, then for every production $B \rightarrow \gamma$ in the grammar, the item $B \rightarrow \bullet\gamma$ must be added to the set. This process is repeated until no new items can be added.
- **Example:** If I contains $E \rightarrow \bullet T$ and $T \rightarrow id$ is a production, then CLOSURE(I) would also include $T \rightarrow \bullet id$.

GOTO()

- The GOTO() operation determines the next state the LR parser will transition to after recognizing a specific grammar symbol (terminal or non-terminal).
- Rule: GOTO(I, X) takes a set of items I (a state) and a grammar symbol X. It identifies all items in I where the dot • is immediately followed by X (e.g., $A \rightarrow \alpha \bullet X \beta$). For each such item, it moves the dot past X (e.g., $A \rightarrow \alpha X \bullet \beta$), collects these new items into a set, and then computes the CLOSURE() of this new set. The resulting set is the target state.
- Example: If I contains $E \rightarrow \bullet T$ and T is a non-terminal, then GOTO(I, T) would result in a new state containing $E \rightarrow T \bullet$ and its closure.

```
SetOfItems CLOSURE(I) {
        J = I;
        repeat
                for ( each item A → α·Bβ in J )
                        for ( each production B → γ of G )
                                if ( B → ·γ is not in J )
                                        add B → ·γ to J;
        until no more items are added to J on one round;
        return J;
}
```

Constructing Canonical Items

We are now ready for the algorithm to construct C, the canonical collection of sets of LR(0) items for an augmented grammar G' — the algorithm is shown in Fig. 4.33.

```
void items(G') {
        C = {CLOSURE({[S' → ·S]})};
        repeat
                for ( each set of items I in C )
                        for ( each grammar symbol X )
                                if ( GOTO(I, X) is not empty and not in C )
                                        add GOTO(I, X) to C;
        until no new sets of items are added to C on a round;
}
```

LR Parsing Algorithm

A schematic of an LR parser is shown in Fig. 4.35. It consists of an input, an output, a stack, a driver program, and a parsing table that has two parts (ACTION and GOTO). The driver program is the same for all LR parsers; only the parsing table changes from one parser to another. The parsing program reads characters from an input buffer one at a time. Where a shift-reduce parser would shift a symbol, an LR parser shifts a *state*. Each state summarizes the information contained in the stack below it.

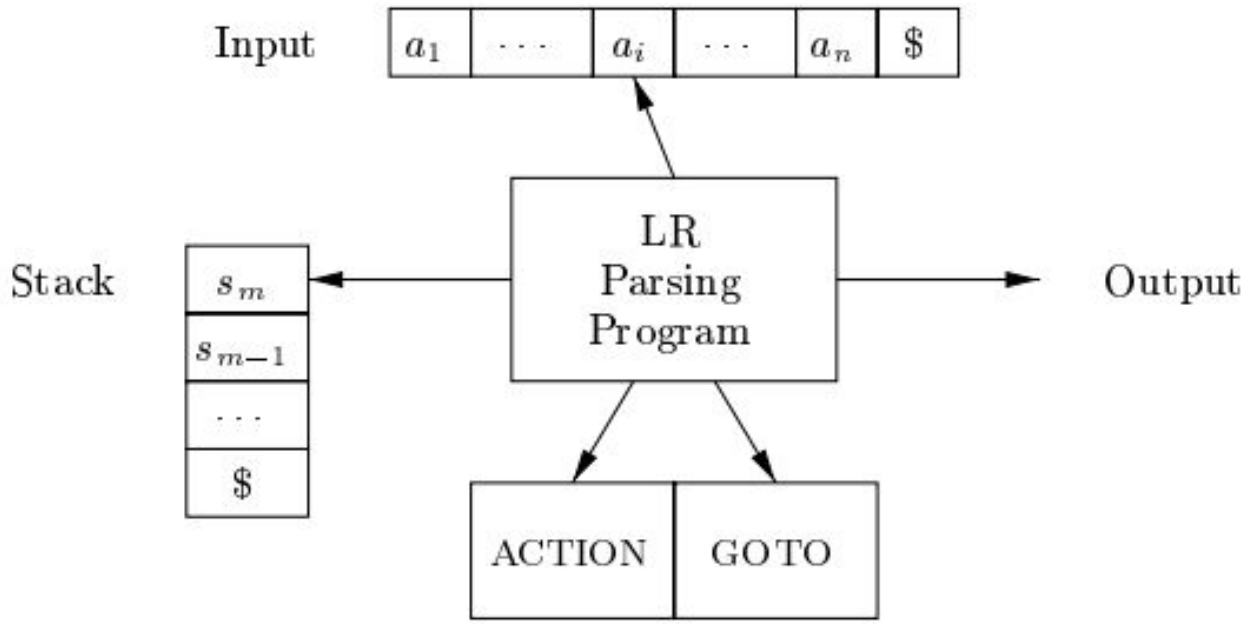


Figure 4.35: Model of an LR parser

Structure of the LR Parsing Table

The parsing table consists of two parts: a parsing-action function ACTION and a goto function GOTO.

1. The ACTION function takes as arguments a state i and a terminal a (or \$, the input endmarker). The value of ACTION$[i, a]$ can have one of four forms:

   (a) Shift j, where j is a state. The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a.

   (b) Reduce $A \to \beta$. The action of the parser effectively reduces β on the top of the stack to head A.

   (c) Accept. The parser accepts the input and finishes parsing.

   (d) Error. The parser discovers an error in its input and takes some corrective action. We shall have more to say about how such error-recovery routines work in Sections 4.8.3 and 4.9.4.

2. We extend the GOTO function, defined on sets of items, to states: if GOTO$[I_i, A] = I_j$, then GOTO also maps a state i and a nonterminal A to state j.

Algorithm 4.44: LR-parsing algorithm.

INPUT: An input string w and an LR-parsing table with functions ACTION and GOTO for a grammar G.

ns ACTION and

OUTPUT: If w is in $L(G)$, the reduction steps of a bottom-up parse for w; otherwise, an error indication.

METHOD: Initially, the parser has s_0 on its stack, where s_0 is the initial state, and $w\$$ in the input buffer. The parser then executes the program in Fig. 4.36.
□

```
let a be the first symbol of w$;
while(1) { /* repeat forever */
        let s be the state on top of the stack;
        if ( ACTION[s, a] = shift t ) {
                push t onto the stack;
                let a be the next input symbol;
        } else if ( ACTION[s, a] = reduce A → β ) {
                pop |β| symbols off the stack;
                let state t now be on top of the stack;
                push GOTO[t, A] onto the stack;
                output the production A → β;
        } else if ( ACTION[s, a] = accept ) break; /* parsing is done */
        else call error-recovery routine;
}
```

Example of LR(0) Parser

- Consider a Grammar.
  - S–>AA
    A–>aA|b
- **STEP 1-** Find augmented grammar –

  The augmented grammar of the given grammar is:-

  S'–>.S [0th production]

  S–>.AA [1st production]

  A–>.aA [2nd production]

  A–>.b [3rd production]

Explanation of Graph (Data Flow Graph)

- In the figure, I0 consists of augmented grammar.
- Io goes to I1 when ‘ . ‘ of 0th production is shifted towards the right of S(S’->S.). This state is the accepted state.
- S is seen by the compiler
- Io goes to I2 when ‘ . ‘ of 1st production is shifted towards the right (S->A.A) . A is seen by the compiler
- I0 goes to I3 when ‘ . ‘ of the 2nd production is shifted towards the right (A->a.A) . a is seen by the compiler.
- I0 goes to I4 when ‘ . ‘ of the 3rd production is shifted towards the right (A->b.) . b is seen by the compiler.
- I2 goes to I5 when ‘ . ‘ of 1st production is shifted towards the right (S->AA.) . A is seen by the compiler

Explanation of Graph (Data Flow Graph)

- I2 goes to I4 when ‘ . ‘ of 3rd production is shifted towards the right (A->b.) . b is seen by the compiler.
- I2 goes to I3 when ‘ . ‘ of the 2nd production is shifted towards the right (A->a.A) . a is seen by the compiler.
- I3 goes to I4 when ‘ . ‘ of the 3rd production is shifted towards the right (A->b.) . b is seen by the compiler.
- I3 goes to I6 when ‘ . ‘ of 2nd production is shifted towards the right (A->aA.) . A is seen by the compiler
- I3 goes to I3 when ‘ . ‘ of the 2nd production is shifted towards the right (A->a.A) . a is seen by the compiler.

- **STEP3 – defining 2 functions: goto[list of non-terminals] and action[list of terminals] in the parsing table**

Explanation of creating table

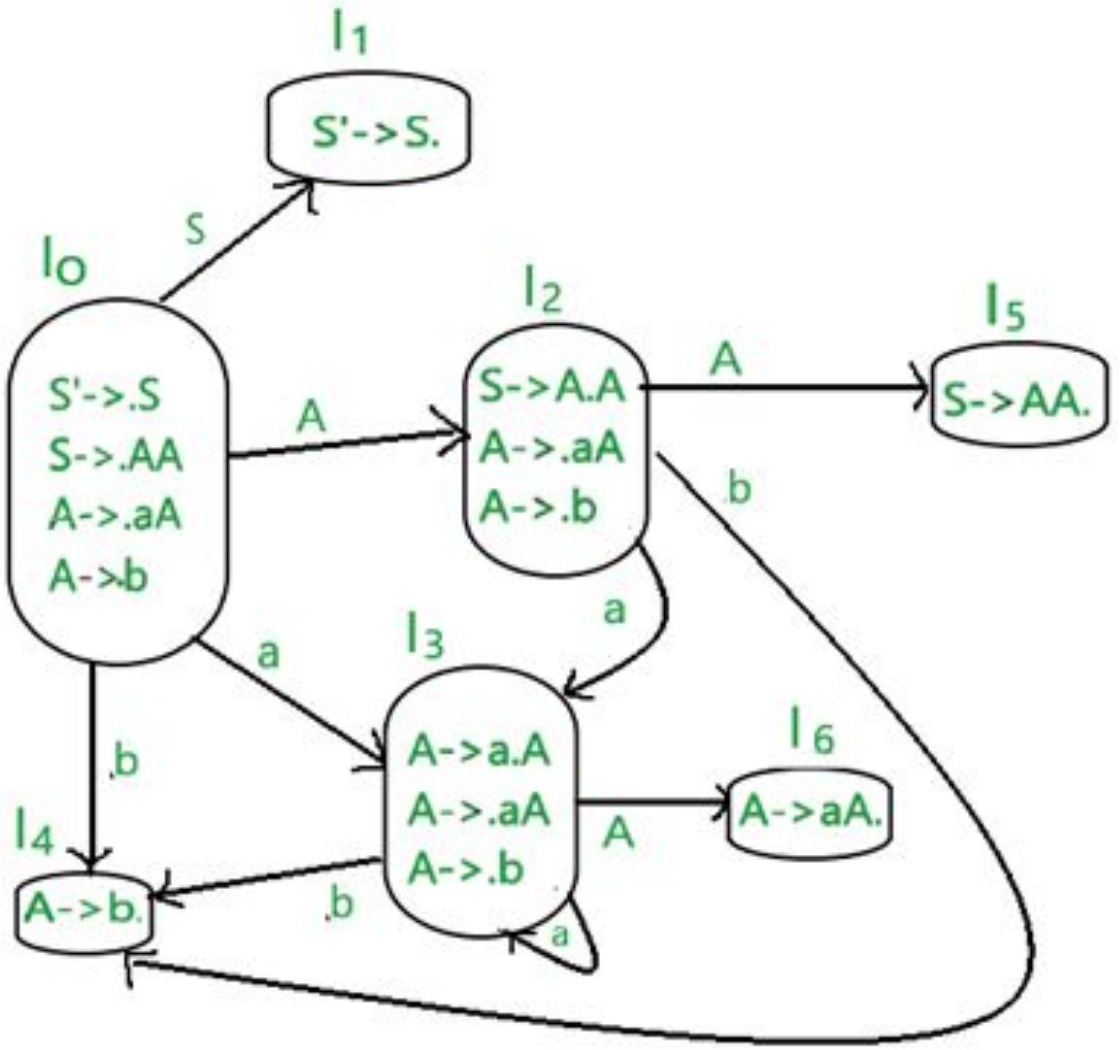
- $ is by default a nonterminal that takes the accepting state.
- 0,1,2,3,4,5,6 denotes I0,I1,I2,I3,I4,I5,I6
- I0 gives A in I2, so 2 is added to the A column and 0 rows.
- I0 gives S in I1, so 1 is added to the S column and 1 row.
- similarly, 5 is written in A column and 2nd row, 6 is written in A column and 3 rows.

Explanation of creating table

- I0 gives an in I3 to .so S3(shift 3) is added to a column and 0 rows.
- I0 gives b in I4, so S4(shift 4) is added to the b column and 0 rows.
- Similarly, S3(shift 3) is added on a column and 2,3 rows, S4(shift 4) is added on b column and 2,3 rows.
- I4 is reduced state as ‘ . ‘ is at the end. I4 is the 3rd production of grammar. So write r3(reduce 3) in terminals.
- I5 is reduced state as ‘ . ‘ is at the end. I5 is the 1st production of grammar. So write r1(reduce 1) in terminals.
- I6 is reduced state as ‘ . ‘ is at the end. I6 is the 2nd production of grammar. So write r2(reduce 2) in terminals.

STEP 2 – Find LR(0) collection of items

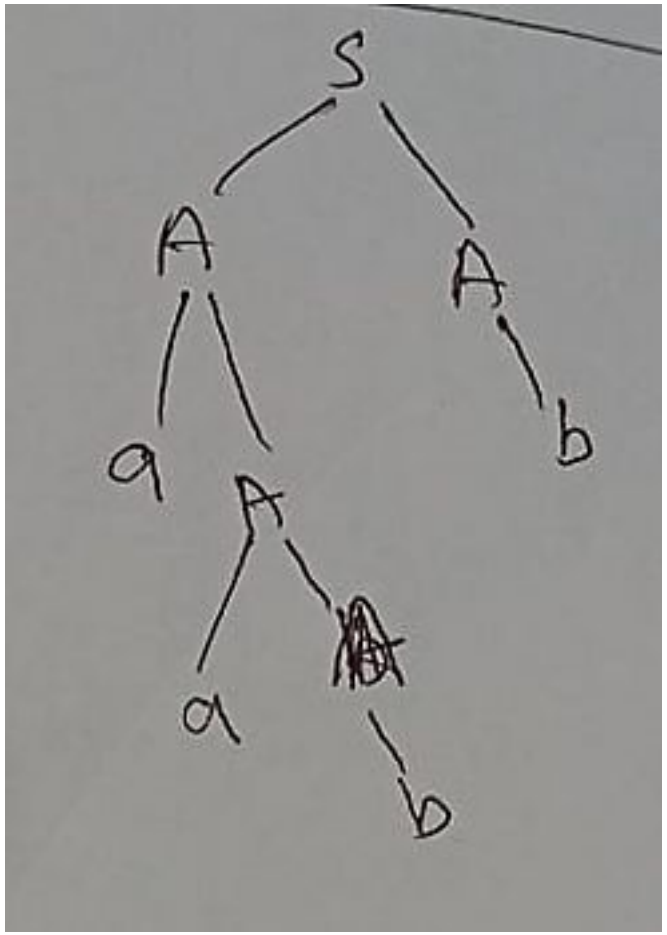
- Below is the figure showing the LR(0) collection of items. We will understand everything one by one.
- The terminals of this grammar are {a,b}
- The non-terminals of this grammar are {S,A}
- **RULE** – if any nonterminal has ‘ . ‘ preceding it, we have to write all its production and add ‘ . ‘ preceding each of its-production.
- **RULE –** from each state to the next state, the ‘ . ‘ shifts to one place to the right.



States	Action			Go to	
	a	b	S	A	S
I_0	S3	S4		2	1
I_1			accept		
I_2	S3	S4		5	
I_3	S3	S4		6	
I_4	r3	r3	r3		
I_5	r1	r1	r1		
I_6	r2	r2	r2		

~~Stack~~ LR(0) Stack implement

Stak	Input	Action
$0	aabb$	shift "a" to stack & goto state 3
$0a3	abb$	shift 'a'
$0a3a3	bb$	shift "b"
$0a3a3b4	b$	A → b
$0a3a3A6	b$	Reduce r3
$0a3a3A6	b$	Reduce r2. A → aA
$0a3A6	b$	Redme R2 A → aA
$0A2	b$	shift b to s4
$0A2b4	$	Reduce r3 A → b
$0A2A5	$	Reduce R1 S → AA
$0S1	$	Accept.



Example 4.45 : Figure 4.37 shows the ACTION and GOTO functions of an LR-parsing table for the expression grammar (4.1), repeated here with the productions numbered:

(1)	$E \to E + T$	(4)	$T \to F$
(2)	$E \to T$	(5)	$F \to (E)$
(3)	$T \to T * F$	(6)	$F \to \mathbf{id}$

The codes for the actions are:

1. si means shift and stack state i,
2. rj means reduce by the production numbered j,
3. acc means accept,
4. blank means error.

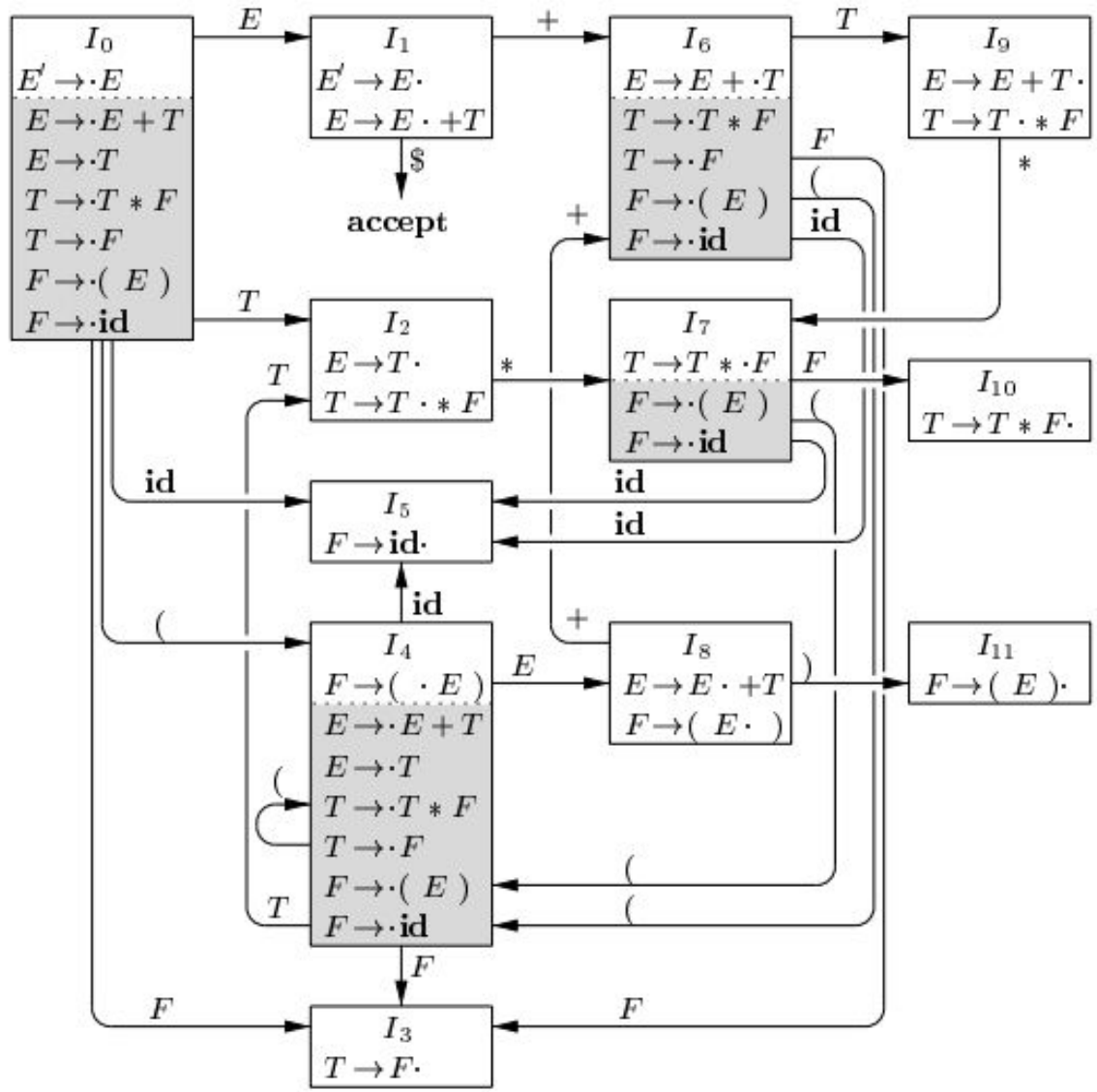


Figure 4.31: LR(0) automaton for the expression grammar (4.1)

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Figure 4.37: Parsing table for expression grammar

	STACK	SYMBOLS	INPUT	ACTION
(1)	0		$\mathbf{id} * \mathbf{id} + \mathbf{id}\,\$$	shift
(2)	0 5	$\mathbf{id}$	$* \mathbf{id} + \mathbf{id}\,\$$	reduce by $F \to \mathbf{id}$
(3)	0 3	F	$* \mathbf{id} + \mathbf{id}\,\$$	reduce by $T \to F$
(4)	0 2	T	$* \mathbf{id} + \mathbf{id}\,\$$	shift
(5)	0 2 7	$T *$	$\mathbf{id} + \mathbf{id}\,\$$	shift
(6)	0 2 7 5	$T * \mathbf{id}$	$+ \mathbf{id}\,\$$	reduce by $F \to \mathbf{id}$
(7)	0 2 7 10	$T * F$	$+ \mathbf{id}\,\$$	reduce by $T \to T * F$
(8)	0 2	T	$+ \mathbf{id}\,\$$	reduce by $E \to T$
(9)	0 1	E	$+ \mathbf{id}\,\$$	shift
(10)	0 1 6	$E +$	$\mathbf{id}\,\$$	shift
(11)	0 1 6 5	$E + \mathbf{id}$	$\$$	reduce by $F \to \mathbf{id}$
(12)	0 1 6 3	$E + F$	$\$$	reduce by $T \to F$
(13)	0 1 6 9	$E + T$	$\$$	reduce by $E \to E + T$
(14)	0 1	E	$\$$	accept

Figure 4.38: Moves of an LR parser on $\mathbf{id} * \mathbf{id} + \mathbf{id}$

Try This Example on LR Parser

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow €$

$B \rightarrow €$

Another Example

$S \rightarrow Aa \mid bAc \mid Bc \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

Another Example

•

$$S \rightarrow L = R \mid R$$
$$L \rightarrow * R \mid id$$
$$R \rightarrow L$$

SLR(1) Parser

- SLR (1) refers to simple LR Parsing. It is same as LR(0) parsing.
- The only difference is in the parsing table.
- To construct SLR (1) parsing table, we use canonical collection of LR (0) item.
- In the SLR (1) parsing, we place the reduce move only in the follow of left hand side.

Various steps involved in the SLR (1) Parsing:

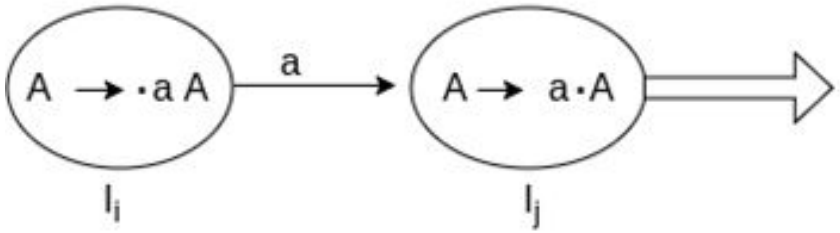
- For the given input string write a context free grammar
- Check the ambiguity of the grammar
- Add Augment production in the given grammar
- Create Canonical collection of LR (0) items
- Draw a data flow diagram (DFA)
- Construct a SLR (1) parsing table

- SLR (1) Table Construction

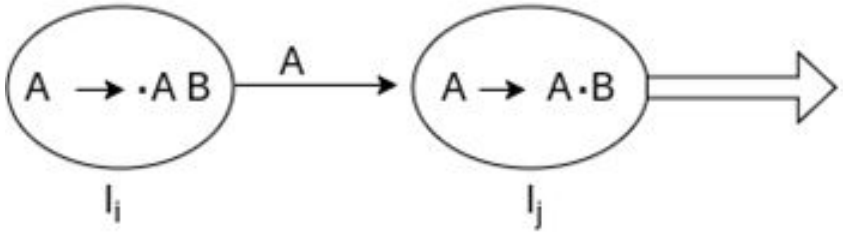
The steps which use to construct SLR (1) Table is given below:

- If a state (Ii) is going to some other state (Ij) on a terminal then it corresponds to a shift move in the action part.
- If a state (I_i) is going to some other state (I_j) on a variable then it correspond to go to move in the Go to part.
- If a state (I_i) contains the final item like A → ab• which has no transitions to the next state then the production is known as reduce production. For all terminals X in FOLLOW (A), write the reduce entry along with their production numbers.

Steps Diagrams



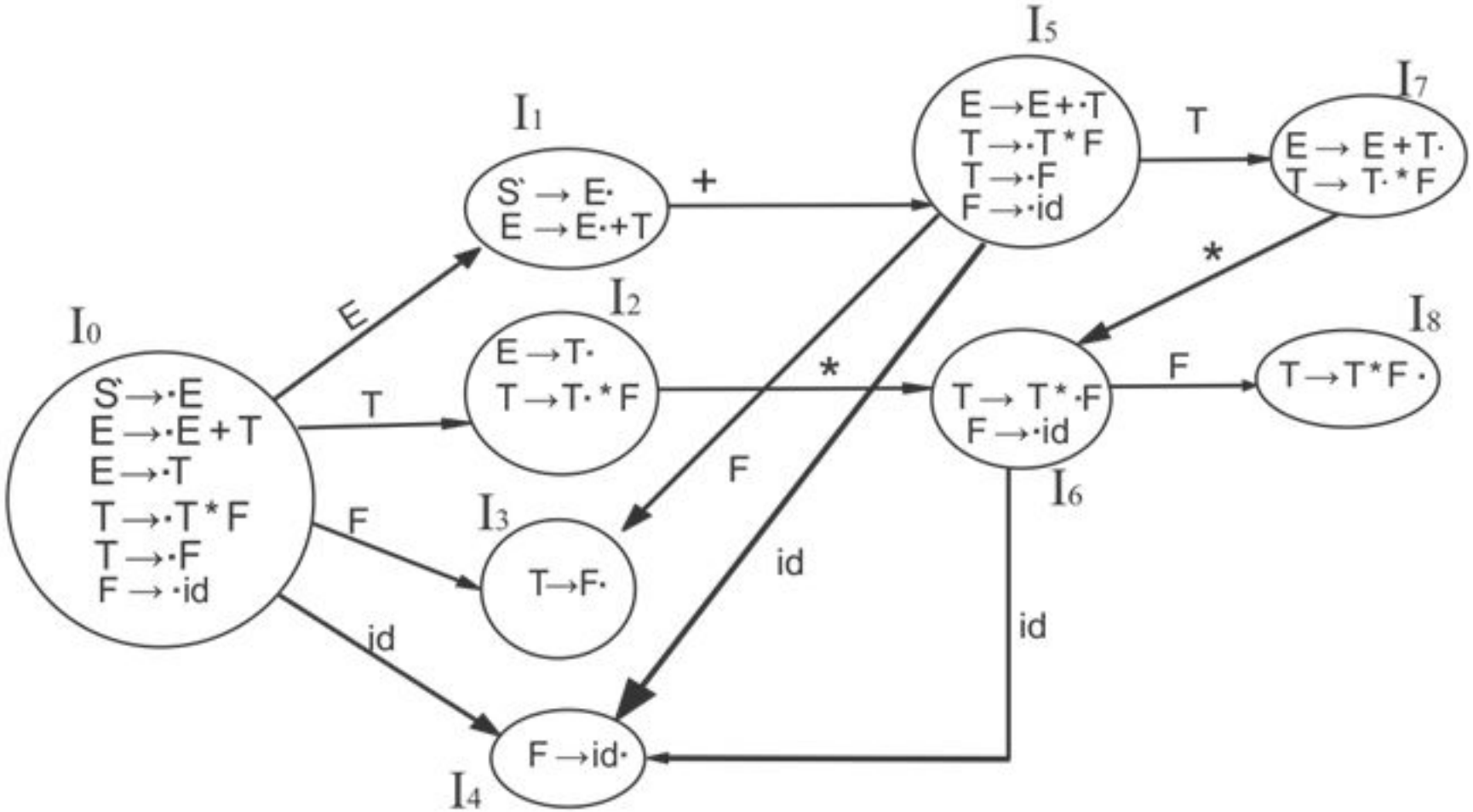
States	Action		Go to
	a	$	A
I_i I_j	Sj		



States	Action		Go to
	a	$	A
I_i I_j			j

Example of SLR(1)

- SLR (1) Grammar
  - S → E
    E → E + T | T
    T → T * F | F
    F → id
- Add Augment Production and insert '•' symbol at the first position for every production in G
  - S` → •E
    E → •E + T
    E → •T
    T → •T * F
    T → •F
    F → •id


I0
S`→·E
E →·E + T
E →·T
T →·T * F
T →·F
F → ·id
I1
S` → E·
E → E·+T
I2
E →T·
T →T· * F
I3
T→F·
I4
F →id·
I5
E →E+·T
T →·T * F
T →·F
F →·id
I6
T→ T * ·F
F →·id
I7
E → E+T·
T → T· * F
I8
T→T*F ·
E
T
F
id
+
*
T
*
F
F
id
id

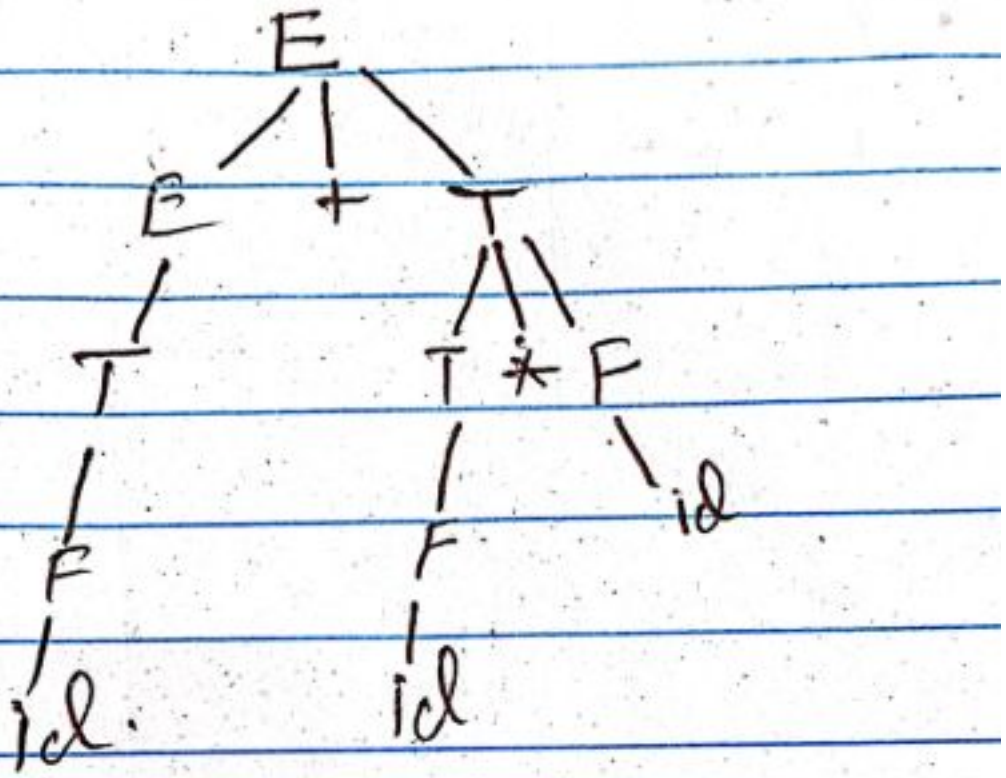
SLR (1) Table

States	Action				Go to		
	id	+	*	**$**	**E**	**T**	**F**
I_0	S_4				1	2	3
I_1		S_5		Accept			
I_2		R_2	S_6	R2			
I_3		R_4	R_4	R4			
I_4		R_5	R5	R5			
I_5	S4					7	3
I_6	S4						8
I_7		R1	S6	R1			
I_8		R3	R3	R3			

SLR Parse tree

Stack	Input	Action
$0	id+id*id$	Shift 5 to stack
$0id4	+id*id$	Reduce F→id (r5)
$0F3	+id*id$	Reduce T→F (R4)
$0T2	+id*id$	Reduce E→T (R2)
$0E1	+id*id$	Shift + to S5
$0E1+5	id*id$	Shift id to S4
$0E1+5id4	*id$	Reduce F→id (R5)
$0E1+5F3	*id$	Reduce T→F (r4)
$0E1+5T7	*id$	Shift * to S6
$0E1+5T7*6	id$	Shift id to S4
$0E1+5T7*6id4	$	Reduce F→id (r5)
$0E1+5T7*6F8	$	Reduce T→T*F (r3)
$0E1+5T7	$	Reduce E→E+T (r1)
$0E1	$	Accept

SLR Parse Tree



Explanation of Table Creation

- First (E) = First (E + T) ∪ First (T)
- First (T) = First (T * F) ∪ First (F)
- First (F) = {id}
- First (T) = {id}
- First (E) = {id}
- Follow (E) = First (+T) ∪ {$} = {+, $}
- Follow (T) = First (*F) ∪ First (F)
- = {*, +, $}
- Follow (F) = {*, +, $}

- I1 contains the final item which drives S → E• and follow (S) = {$}, so action {I1, $} = Accept
- I2 contains the final item which drives E → T• and follow (E) = {+, $}, so action {I2, +} = R2, action {I2, $} = R2
- I3 contains the final item which drives T → F• and follow (T) = {+, *, $}, so action {I3, +} = R4, action {I3, *} = R4, action {I3, $} = R4

- I4 contains the final item which drives F → id• and follow (F) = {+, *, $}, so action {I4, +} = R5, action {I4, *} = R5, action {I4, $} = R5
- I7 contains the final item which drives E → E + T• and follow (E) = {+, $}, so action {I7, +} = R1, action {I7, $} = R1
- I8 contains the final item which drives T → T * F• and follow (T) = {+, *, $}, so action {I8, +} = R3, action {I8, *} = R3, action {I8, $} = R3

Conflicts

- RR- Reduce Reduce Conflicts
- A grammar
  - A->a
    B->b
    C->aB
- Augmented Grammar
  - S->.A
    A->.a
    B->.b
    C->.aB

Reference:

- Compilers: Principles, Techniques, and Tools 2nd Edition
  by Alfred Aho , Jeffrey Ullman, Ravi Sethi, Monica Lam

- Introduction to Compilers and Language Design | Second Edition |
  Prof. Douglas Thain - University of Notre Dame